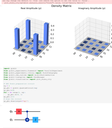
HW set 1 Problem #5

```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_5")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```




```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_5")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```

```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_5")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```

```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_5")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```

```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```


HW set 1 Problem #6

```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_6")
qc.h(0)
qc.cx(0, 1)
qc.rz(pi/4, 1)
qc.cx(0, 1)
qc.h(0)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```

```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_6")
qc.h(0)
qc.cx(0, 1)
qc.rz(pi/4, 1)
qc.cx(0, 1)
qc.h(0)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_6")
qc.h(0)
qc.cx(0, 1)
qc.rz(pi/4, 1)
qc.cx(0, 1)
qc.h(0)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```

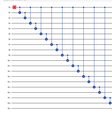
I only do the above step because the rest of the circuit is just a bunch of cancelling gates (controlled-Z and controlled-Rz). Everything is just equivalent to the first circuit, which has the same

```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_6")
qc.h(0)
qc.cx(0, 1)
qc.rz(pi/4, 1)
qc.cx(0, 1)
qc.h(0)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



HW set 1 Problem #7

```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```



```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from numpy import pi, sqrt, exp, cos, sin
from numpy.linalg import norm

qc = QuantumCircuit(2, 2, name="HW1_7")
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl', style='iqp')
```



```
sim = AerSimulator(method='statevector')
qc.save_statevector()
result = sim.run(qc).result()
sv = result.get_statevector()
print("Statevector:", sv)

from qiskit.visualization import plot_bloch_multivector, plot_histogram, plot_state_city
plot_state_city(sv)
```

```
Statevector: Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j])
```